

Secure LLM Assistant

Programming Project Report

Author: Ville Viinikka

Repository: VIINIKKA/secure-programming-llm

Contents

1. General description	3
1.1 Purpose of the program.....	3
1.2 How the program is used	3
1.3 User interface description.....	3
2. Structure of the program.....	5
2.1 Runtime architecture	5
2.2 Backend structure	5
2.3 Frontend structure.....	5
2.4 Deployment and DevSecOps structure.....	6
3. Secure programming solutions	7
3.1 Selected checklist	7
3.2 OWASP LLM Top 10 mapping	7
3.3 Authentication and session security	8
3.4 MFA security	8
3.5 Input validation and prompt security	8
3.6 PII protection and logging.....	9
3.7 Output handling.....	9
3.8 Rate limiting and resource controls	9
3.9 Supply chain and CI security	9
3.10 Code comments.....	9
4. Earlier work and changes	9
5. Testing	11
5.1 Automated testing	11
5.2 Manual security testing checklist	11
5.3 Findings and fixes based on testing	12
5.4 Testing limitations	12
6. Not yet implemented.....	12
7. Known security issues and residual vulnerabilities	13
7.1 Default secrets	13
7.2 Token storage in browser sessionStorage.....	13
7.3 Streaming redaction edge case	13
8. Suggestions for improvement	13

1. General description

1.1 Purpose of the program

Secure LLM Assistant is a local large language model assistant application. The main purpose is to provide a chat interface to a locally running Ollama model while applying secure programming principles before any user input reaches the model. The program is therefore not only a generic chatbot but it is a secure systems project where the backend is the controlled trust boundary between the browser and the model runtime.

The central security idea is that the user never communicates directly with Ollama. The request flow is:

User browser -> React frontend served by Nginx -> FastAPI backend security gateway -> Ollama local model runtime

The backend enforces authentication, authorization, prompt sanitization, prompt injection filtering, personal data redaction, rate limits, input token limits, output token limits, and timeout handling before communicating with the model.

1.2 How the program is used

The application is started with Docker Compose. A typical local start is:

```
cp .env.example .env
docker compose up -d --build
```

After startup the user opens the frontend in a browser. The frontend exposes the login, registration, optional two factor authentication setup, and chat views. A user must authenticate before using the chat endpoints.

The main usage flow is:

1. Start the containerized stack.
2. Open the frontend.
3. Register an account or log in with an existing account.
4. Optionally start two factor authentication setup.
5. Send a prompt through the chat interface.
6. The frontend sends the prompt to the backend.
7. The backend validates the session, applies security controls, forwards the sanitized prompt to Ollama, redacts the response if required, and returns the result.

1.3 User interface description

The user interface is intentionally simple. Its purpose is to make the protected chat flow clear rather than to demonstrate a complex UI.

The main user interface states are:

1. Sign in view. The user enters a username and password. If the account has two factor authentication enabled, the interface requests a one time code from an authenticator application.
2. Create account view. The user can create a local account when self signup is enabled. Password rules are enforced by the backend.
3. Two factor authentication setup view. The user can generate a TOTP secret, scan a QR code, enter the one time code, and enable MFA.

4. Protected chat view. Authenticated users can send prompts and receive streamed or normal model responses.
5. Security status area. The UI displays the MFA status and provides controls for enabling or disabling MFA.

The model output is rendered as plain React text. The frontend does not use raw HTML insertion for assistant messages. This is important because model output must be treated as untrusted data.

2. Structure of the program

2.1 Runtime architecture

The runtime contains three main services:

1. frontend: React application served by Nginx.
2. backend: FastAPI service that acts as the security gateway.
3. ollama: Local LLM runtime.

The backend also uses a SQLite based authentication and session store. The store contains users, password hashes, MFA state, active refresh sessions, session expiry, and revocation state.

2.2 Backend structure

File	Purpose
backend/app/main.py	FastAPI endpoint. Defines health, authentication, MFA, refresh, logout, chat, and streaming chat routes. Connects request validation, authentication, authorization, sanitization, PII redaction, rate limiting, token guardrails, and the LLM service.
backend/app/auth.py	Authentication and session implementation. Contains password hashing, login, registration, JWT creation, refresh token rotation, logout revocation, TOTP MFA, server side session validation, and scope checks.
backend/app/security.py	Input and output security helpers. Contains control character stripping, whitespace normalization, prompt injection pattern checks, PII redaction, log safe sanitization, and approximate token counting.
backend/app/llm_service.py	Ollama client wrapper. Builds Ollama requests, applies request timeouts, handles upstream errors, parses streaming JSON lines, and isolates model communication from route code.
backend/app/config.py	Runtime configuration through environment variables.

2.3 Frontend structure

File	Purpose
frontend/src/App.jsx	Top level React component. Selects between authentication loading, login or registration, and authenticated chat views.
frontend/src/hooks/useSecureLlmApp.js	Main frontend state and API hook. Handles login, registration, refresh, logout, session restoration, MFA setup, and chat streaming.
frontend/src/components/ChatView.jsx	Chat screen and composer.
frontend/src/components/MessageList.jsx	Renders user and assistant messages as text.
frontend/src/lib/api.js	API response parsing and SSE event parsing.
frontend/src/lib/session.js	Session storage helpers.

2.4 Deployment and DevSecOps structure

Deployment is defined with Docker Compose. The backend and frontend containers use hardened settings such as read only filesystems, temporary writable directories, dropped capabilities, no new privileges, PID limits, and health checks.

The Jenkins pipeline performs the following stages:

1. Checkout.
2. Backend tests and static analysis.
3. Python dependency audit.
4. Frontend tests and npm audit.
5. Container build.
6. Staging deployment on the master-staging branch.
7. Post deployment health check.

Dependabot is configured for backend Python dependencies, frontend npm dependencies, and Docker base image updates.

3. Secure programming solutions

3.1 Selected checklist

The main checklist used for this report is OWASP Top 10 for Large Language Model Applications 2025. This checklist fits the project because the application is specifically an LLM based system. Traditional secure programming principles are also applied where relevant, especially authentication, authorization, input validation, session management, least privilege, dependency management, and secure deployment.

3.2 OWASP LLM Top 10 mapping

Checklist item	Status	Implemented solution	Code areas
LLM01 Prompt Injection	Partly implemented	The backend sanitizes input and blocks obvious prompt injection patterns before contacting Ollama.	backend/app/security.py, backend/app/main.py
LLM02 Sensitive Information Disclosure	Baseline implemented	Common PII patterns are redacted before the LLM call, after the response, and before suspicious prompts are written to logs.	backend/app/security.py, backend/app/main.py
LLM03 Supply Chain	Partly implemented	Dependencies are pinned, Jenkins runs pip-audit and npm audit, and Dependabot is configured.	backend/requirements.txt, frontend/package.json, Jenkinsfile, .github/dependabot.yml
LLM04 Data and Model Poisoning	Not applicable currently	The application does not train models or ingest external knowledge bases.	Not applicable
LLM05 Improper Output Handling	Partly implemented	Assistant output is rendered as plain text in React. The application does not evaluate or render model output as HTML.	frontend/src/components/MessageList.jsx
LLM06 Excessive Agency	Implemented by architecture	The model has no tools, no file system access, no shell access, and no ability to perform external actions.	backend/app/llm_service.py
LLM07 System Prompt Leakage	Partly implemented	The backend uses a system prompt and blocks simple attempts to reveal internal instructions. This is not a complete defense against advanced jailbreaks.	backend/app/config.py, backend/app/security.py

LLM08 Vector and Embedding Weaknesses	Not applicable currently	The application does not use a vector database or retrieval augmented generation.	Not applicable
LLM09 Misinformation	Not implemented	The application includes an AI disclaimer, but there is no fact checking, source citation, or confidence scoring.	README.md, SECURITY.md
LLM10 Unbounded Consumption	Baseline implemented	Rate limiting, input token limits, output token caps, request timeout, and container resource restrictions are used.	backend/app/main.py, backend/app/config.py, backend/app/llm_service.py, docker-compose.yml

3.3 Authentication and session security

The application uses local username and password authentication. Passwords are not stored in plaintext. The backend hashes passwords with PBKDF2 HMAC SHA256 using a random salt and uses constant time comparison when verifying password hashes.

Authentication uses short lived access tokens and longer lived refresh tokens. The implementation does not rely on stateless JWTs alone. Each login creates a server side session record in SQLite. Protected routes validate that the access token belongs to an active server side session. This allows logout revocation and session invalidation.

Refresh tokens are rotated. When a refresh token is used, the backend stores a hash of the next refresh token identifier and rejects reuse of old refresh token identifiers. This reduces the impact of token replay.

Scope based authorization is also implemented. The normal chat endpoint requires chat:write, while streaming chat requires chat:stream. The backend trusts role and scope values from the server side session instead of trusting the token body alone.

3.4 MFA security

The program supports optional TOTP based two factor authentication. The MFA setup process stores a pending secret first. MFA is only enabled after the user proves that the authenticator application can generate a valid one time code. Disabling MFA also requires a valid one time code.

This protects user accounts against password compromise better than password only authentication. However, TOTP secrets are currently stored in the local SQLite database. If the project were deployed in a higher risk environment, MFA secrets should be encrypted at rest.

3.5 Input validation and prompt security

FastAPI and Pydantic are used for request validation. Usernames have length and character constraints. Prompts have length constraints. Output token requests are bounded.

The prompt is sanitized before further processing. Control characters are removed, whitespace is normalized, and empty sanitized input is rejected. Obvious jailbreak and prompt leakage patterns are blocked before the LLM is called.

The prompt injection filter is intentionally a baseline control. It catches direct attempts such as asking the model to ignore previous instructions or reveal the system prompt. It is not a complete solution against advanced adversarial prompting.

3.6 PII protection and logging

The backend redacts common sensitive data types such as email addresses, phone numbers, social security number patterns, and credit card number patterns. Redaction is applied to input before it leaves the backend and also to model output before returning to the frontend.

Logs avoid writing raw prompts directly. Suspicious prompts are sanitized for logging and truncated to a maximum length. This reduces the chance that sensitive user input ends up in application logs.

3.7 Output handling

Model output is handled as untrusted data. The frontend renders messages as React text nodes instead of injecting HTML. This protects against many common client side output handling mistakes, especially if the model emits text that looks like HTML or script code.

3.8 Rate limiting and resource controls

The backend uses request rate limiting. It also applies approximate input token limits, output token limits, and LLM request timeouts. The Ollama client uses configured timeout and connection limits.

The Docker Compose configuration adds runtime restrictions. The backend and frontend use read only filesystems, limited temporary writable directories, dropped capabilities, no new privileges, PID limits, and health checks.

3.9 Supply chain and CI security

Backend dependencies are pinned in backend/requirements.txt. Frontend dependencies are managed through npm and package-lock.json. Jenkins runs Python tests, Bandit static analysis, pip-audit, frontend tests, and npm audit before building containers and deploying to staging.

Dependabot is configured to check Python, npm, and Docker dependencies weekly. This helps keep the project maintainable after submission.

3.10 Code comments

Security related comments are present in the code around important decisions, including constant time password comparison, pending MFA secret handling, refresh token rotation, redaction before contacting the model, streaming response parsing, and session restoration behavior. These comments document why the implementation is structured in a specific way rather than only describing what the code does.

4. Earlier work and changes

This project is implemented as a dedicated secure programming project. It uses standard frameworks and tools, including FastAPI, React, Nginx, Docker Compose, SQLite, PyJWT, pyotp, Ollama, Jenkins, Bandit, pip-audit, npm audit, and Dependabot.

The repository also states that language models were used during development as coding assistants. Final implementation choices, integration, testing, and repository contents remained under project control.

Important project changes visible from the repository history include:

1. Addition of frontend tests and stronger CI security gates.
2. Pinning of frontend test tooling to support the Jenkins Node environment.
3. Addition and revision of project documentation and presentation material.

5. Testing

5.1 Automated testing

The backend test suite covers security critical behavior. Existing tests include authentication behavior, registration, weak password rejection, duplicate username rejection, TOTP MFA setup and disable flow, bearer token requirement for protected chat, successful authenticated chat, prompt sanitization, prompt injection detection, PII redaction, log truncation, and approximate token counting.

The pipeline also runs static and dependency security checks:

```
pytest -q backend/tests
bandit -q -r backend/app
pip-audit -r backend/requirements.txt
npm --prefix frontend run test:run
npm --prefix frontend audit --audit-level=high
```

5.2 Manual security testing checklist

The following manual tests should be performed before final submission. The expected result documents the intended secure behavior.

Test	Manual step	Expected secure result	Status
Unauthenticated chat access	Send POST /api/chat without an Authorization header.	Backend returns 401 and does not contact Ollama.	To be executed before final submission
Invalid login	Try logging in with a valid username and wrong password.	Backend returns 401 and no token pair is issued.	To be executed before final submission
Weak password registration	Try registering with a short password or a password without digits.	Registration is rejected.	To be executed before final submission
Duplicate username	Register the same username twice.	Second registration is rejected.	To be executed before final submission
MFA enable flow	Start MFA setup, scan QR code, enter valid TOTP code.	MFA becomes enabled only after the valid code.	To be executed before final submission
MFA login requirement	Log out and log in with an MFA enabled account without OTP.	Login is rejected and the UI asks for the authenticator code.	To be executed before final submission
Refresh rotation	Use a refresh token, then try to reuse the old refresh token.	Old refresh token is rejected.	To be executed before final submission
Logout revocation	Log out and try to use the old refresh token.	Refresh token is rejected.	To be executed before final submission
Prompt injection baseline	Send a prompt asking to ignore previous instructions and reveal the system prompt.	Request is rejected by backend policy before Ollama is called.	To be executed before final submission
PII redaction	Send a prompt containing an email address and a US SSN pattern.	Sensitive values are replaced with redaction placeholders before model access.	To be executed before final submission

Large input	Send a prompt larger than the configured input token limit.	Backend rejects the request.	To be executed before final submission
XSS output handling	Ask the model to output script like HTML.	Output is displayed as text, not executed as script.	To be executed before final submission
Direct Ollama exposure	From outside the Docker network, try to access Ollama on port 11434.	Ollama is not directly exposed by Compose.	To be executed before final submission
Health check	Run curl http://localhost:8080/healthz.	The frontend proxy and backend return a successful health response.	To be executed before final submission

5.3 Findings and fixes based on testing

The repository history indicates that the CI process was improved by adding frontend tests and stronger security gates. The frontend test toolchain was also pinned to work with the Jenkins Node 20.18 environment. This is a good example of a testing finding that led to a practical maintainability fix.

The current test coverage also confirms that several important controls were implemented intentionally, including password policy enforcement, MFA flow behavior, bearer token protection, prompt injection detection, and PII redaction.

5.4 Testing limitations

1. The prompt injection tests cover obvious patterns only.
2. The current PII redaction logic is regex based and does not detect every possible sensitive data type.
3. Streaming output redacts each chunk separately, which may miss sensitive values split across stream chunks.
4. TLS, reverse proxy behavior, browser security headers, and real deployment network exposure require environment specific testing.
5. Load testing and abuse testing have not been fully implemented.

6. Not yet implemented

1. External identity provider integration, SSO, and centralized user management are not implemented.
2. Model output fact checking and citation based verification are not implemented.
3. Browser security headers are not fully configured in the provided Nginx template.
4. Account lockout and per account login throttling are not fully implemented.
5. No vector database, RAG pipeline, or data poisoning defenses are implemented because the current system does not use those features.

7. Known security issues and residual vulnerabilities

7.1 Default secrets

The example configuration contains default values such as `change-me` and `change-me-jwt-secret`. These are acceptable as examples but must never be used in deployment. The application should fail startup in production if default secrets are detected.

7.2 Token storage in browser `sessionStorage`

The frontend stores access and refresh tokens in `sessionStorage`. This is better than long term `localStorage` persistence, but JavaScript readable storage is still exposed if a future XSS issue is introduced. A more secure production design would store refresh tokens in `Secure`, `HttpOnly`, `SameSite` cookies and keep access tokens in memory.

7.3 Streaming redaction edge case

Streaming responses are redacted per chunk. If a sensitive value is split across stream chunks, the regex redactor may not detect it before partial output is sent to the browser. A safer implementation would use a rolling buffer or delayed redaction window for streaming output.

8. Suggestions for improvement

1. Add production mode startup checks that reject default secrets and insecure public deployment settings.
2. Move refresh tokens to `Secure`, `HttpOnly`, `SameSite` cookies and keep access tokens in memory.
3. Add account based login throttling and temporary lockout after repeated failed attempts.
4. Pin Docker images by immutable digest in release branches.
5. Add more adversarial prompt injection tests, including multilingual and encoded bypass attempts.
6. Add structured security event logging for login failures, token refresh failures, MFA changes, blocked prompts, and rate limit events.
7. Add frontend tests for login flow, MFA flow, streaming chat behavior, and safe rendering of HTML like model output.